

Cross Validation and Regularisation

Goodness of MLE for Linear Regression

Consider a dataset

$$\{(x_1, y_1) (x_2, y_2) \dots (x_n, y_n)\} \mid x \in \mathbb{R}^d ; y \in \mathbb{R}$$

The probabilistic view of linear regression assumes that the target variable y_i can be modeled as a linear combination of the input features x_i , with an additional noise

term ϵ following a zero-mean Gaussian distribution with variance σ^2 . Mathematically, this can be expressed as:

$$y \mid x = w^T x + \epsilon$$

Assuming $\epsilon \sim \mathcal{N}(0, \sigma^2)$

$$y \mid x \sim \mathcal{N}(w^T x, \sigma^2)$$

To estimate the weight vector w that best fits the data, we can apply the principle of Maximum Likelihood (ML). The ML estimation seeks to find the parameter values that maximize the likelihood of observing the given data. Assuming that the noise term ϵ follows a zero-mean Gaussian distribution with variance σ^2 , we can express the likelihood function as:

$$\mathcal{L}(w, x_1, \dots, x_n, y_1, \dots, y_n) = \prod_{i=1}^n e^{\frac{-(w^T x_i - y_i)^2}{2\sigma^2}} \cdot \frac{1}{\sigma\sqrt{2\pi}}$$

$$\log(\mathcal{L}(w, x_1, \dots, x_n, y_1, \dots, y_n)) = \sum_{i=1}^n \frac{-(w^T x_i - y_i)^2}{2\sigma^2} \cdot \frac{1}{\sigma\sqrt{2\pi}}$$

$$\max_w \sum_{i=1}^n -(w^T x_i - y_i)^2 \equiv \min_w \sum_{i=1}^n (w^T x_i - y_i)^2$$

$$\hat{w}_{ml} \equiv w^* = (XX^T)^{\dagger} (Xy)$$

This expression is equivalent to the sum squared error (SSE) objective function used in linear regression. Therefore, finding the maximum likelihood estimate $\hat{w}_{ml}$ assuming zero mean Gaussian noise is equivalent to solving the linear regression problem using the squared error loss.

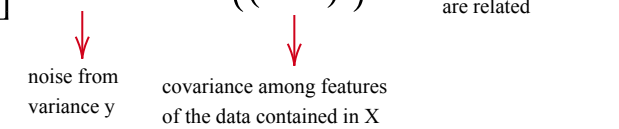
To evaluate the quality of the estimated parameters, we measure the mean squared error (MSE) between the estimated parameters and the true parameters w

The MSE is given by:

$$\mathbb{E} \left[\|\hat{w}_{ml} - w\|_2^2 \right]$$

Interestingly, the MSE can be expressed as:

$$\mathbb{E} \left[\|\widehat{w}_{ml} - w\|_2^2 \right] = \sigma^2 \times \text{trace} \left((XX^T)^{\dagger} \right)$$



noise from variance y covariance among features of the data contained in X implies how features are related

Cross Validation for Minimizing MSE

$$\text{trace}\left((XX^T)^+\right)$$

For any Matrix $A_{d \times d}$,

$$\text{trace}(A) = \sum_{i=1}^d a_i = \sum_{i=1}^d \lambda_i \quad \rightarrow \text{Eigenvalues of A}$$

$\downarrow$
diagonal entries
of A

Let, Eigen Values of (XX^T) be $\{\lambda_1, \lambda_2, \dots, \lambda_d\}$

Eigen Values of $(XX^T)^{-1}$ be $\left\{\frac{1}{\lambda_1}, \frac{1}{\lambda_2}, \dots, \frac{1}{\lambda_d}\right\}$

Mean Squared Error can now be expressed as

$$\mathbb{E}\left[\|\widehat{w}_{ml} - w\|_2^2\right] = \sigma^2 \times \left(\sum_{i=1}^d \frac{1}{\lambda_i}\right)$$

To Improve the estimator we added a new term λI and modified the estimator to $\hat{w}_{new}$

$$\hat{w}_{new} = (XX^T + \lambda I)^{-1} Xy$$

Here $\lambda \in \mathbb{R}$ and $I \in \mathbb{R}^{d \times d}$ represents a identity matrix

For some matrix A, let eigenvalues be $\{\lambda_1, \lambda_2, \dots, \lambda_d\}$

$\therefore$ eigenvalues of $(A + \lambda I)$ be $\{\lambda_1 + \lambda, \lambda_2 + \lambda, \dots, \lambda_d + \lambda\}$

$$\text{trace}((XX^T + \lambda I)^{-1}) = \sum_{i=1}^d \frac{1}{\lambda_i + \lambda}$$

Why do we add λI ?

The MSE error depends upon variance and trace, we aim to reduce the value of trace to reduce MSE Overall, thus if we increase the value in the denominator, its reciprocal will become small value.

Existence Theorem

According to the Existence Theorem, there exists a value of λ such that $\hat{w}_{new}$ exhibits a lower mean squared error than $\hat{w}_{ml}$. In practice, the value for λ is determined using cross-validation techniques.

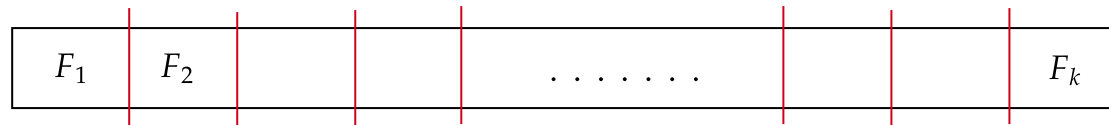
Training-Validation Split: The training set is randomly divided into a training set and a validation set, typically in an 80:20 ratio. Train on the

training set and check for error in validation set. Among various λ values, the one that yields the lowest error is selected.



But what is the right way to split into 80:20? we might split it random but we might get unlucky and the 20% in validation set might not be the good representative of what we are learning from the 80% training set, so the error might not be true error. To avoid this, we use K-Fold cross validation

K-Fold Cross Validation: The training set is partitioned into K equally-sized parts. The model is trained K times, each time using K-1 parts as the training set and the remaining part as the validation set. The λ value that leads to the lowest average error is chosen.



- Train on Folds

$$\{F_1, \dots, F_{i-1}, F_{i+1}, \dots, F_k\}$$

- Validate on F_i
- Pick λ that gives the least average error

How to choose K ?

The Larger the value of K, the Better. In extreme cases, you can use LOOCV.

Leave-One-Out Cross Validation: The model is trained using all but one sample in the training set, and the left-out sample is used for validation. This process is repeated for each sample in the dataset. The optimal λ is determined based on the least average error across all iterations.

By employing cross-validation techniques, we can enhance the performance of the linear regression model by selecting an appropriate value of λ

Bayesian Modeling for Linear Regression

Alternative way to look at $\hat{w}_{ml}$ is Bayesian Modeling. In Bayesian Modeling, we have a prior over the parameter we want to estimate and then we have likelihood, based on this we get a posterior.

Here our parameter of estimate is w , we need a prior on w , i.e., $p(w)$ and likelihood is given as

$$y | x \sim \mathcal{N}(w^T x, 1)$$

Therefore a choice of prior will be Normal as well

$$w \sim \mathcal{N}(0, \gamma^2 I)$$

where $\gamma^2 I$ is a matrix of shape $d \times d$

As Usual,

$$\underbrace{P(w | \{(x_1, y_1) \dots (x_n, y_n)\})}_{\text{Posterior}} \propto \underbrace{P(\{(x_1, y_1) \dots (x_n, y_n)\} | w)}_{\text{Likelihood}} \cdot \underbrace{P(w)}_{\text{Prior}}$$

$$\begin{aligned}
& \propto \left(\prod_{i=1}^n e^{-\frac{(y_i - w^T x_i)^2}{2(1)}} \right) \cdot \left(\prod_{i=1}^d e^{-\frac{(w_i - 0)^2}{2\gamma^2}} \right) \\
& \propto \left(\prod_{i=1}^n e^{-\frac{(y_i - w^T x_i)^2}{2}} \right) \cdot \left(e^{-\sum_{i=1}^d \frac{(w_i)^2}{2\gamma^2}} \right) \\
& \propto \left(\prod_{i=1}^n e^{-\frac{(y_i - w^T x_i)^2}{2}} \right) \cdot \left(e^{-\frac{\|w\|^2}{2\gamma^2}} \right)
\end{aligned}$$

How the Maximum Aposteriori Estimation will look like ?

$$\begin{aligned}
\hat{w}_{map} &= \arg \max_w \sum_{i=1}^n -\frac{(y_i - w^T x_i)^2}{2} - \frac{\|w\|^2}{2\gamma^2} \\
\hat{w}_{map} &= \arg \min_w \frac{1}{2} \sum_{i=1}^n (y_i - w^T x_i)^2 + \frac{1}{2\gamma^2} \|w\|^2
\end{aligned}$$

Take derivative w.r.t w and equate it to 0

$$\begin{aligned}\nabla f(w) &= (XX^T)w - Xy + \frac{w}{\gamma^2} \\ \therefore \hat{w}_{map} &= \left(XX^T + \frac{1}{\gamma^2} I \right)^{-1} Xy\end{aligned}$$

Conclusion, MAP Estimator for linear regression with a Gaussian prior $\mathcal{N}(0, \gamma^2 I)$ for w is equivalent to new estimator $\hat{w}_{new}$ we discussed earlier

$$\hat{w}_{new} = (XX^T + \lambda I)^{-1} Xy$$

Ridge Regression

$$\hat{w}_{ml} = \arg \min_w \sum_{i=1}^n (w^T x_i - y_i)^2$$
$$\hat{w}_R = \arg \min_w \underbrace{\sum_{i=1}^n (w^T x_i - y_i)^2}_{\text{Loss}} + \underbrace{\lambda \|w\|_2^2}_{\text{Regularizer}}$$

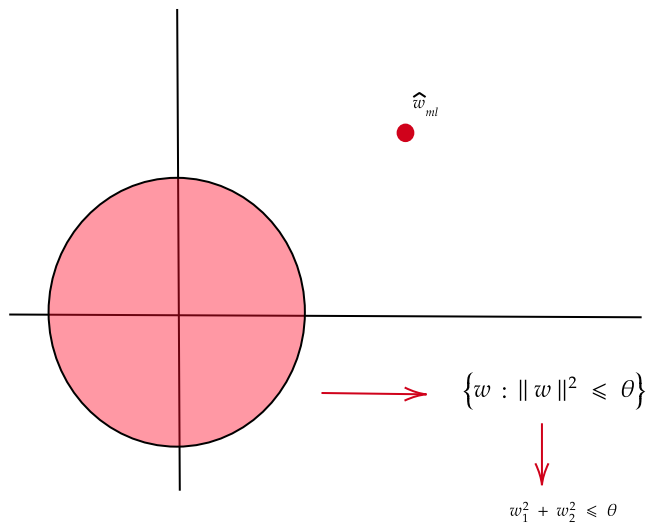
↓
Ridge

- Ridge regression is a linear regression technique that addresses multicollinearity and overfitting by adding a penalty term to the ordinary least squares method.
- Regularizer is used for penalizing the length of w , and pushing it as close to zero.
- Whereas, Loss function is trying to find the w for which the loss is as small as possible.
- In case of redundant features in a dataset, Ridge Regression will choose those features that have least weight

Relation Between Linear Regression and Ridge Regression

$$\hat{w}_R = \arg \min_w \sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \|w\|_2^2 \quad (1)$$
$$\equiv$$
$$\min_{w \in \mathbb{R}^d} \sum_{i=1}^n (w^T x_i - y_i)^2 \text{ s.t. } \|w\|^2 \leq \theta \rightarrow \text{depends upon } \lambda$$

Here, the value of θ depends on λ For every choice of $\lambda > 0$, $\exists \theta$ s.t. optimal solution of (1) *and* (2) Coincide



What is the loss/error of linear regression $\hat{w}_{ml}$

$$\min_{w \in \mathbb{R}^d} \sum_{i=1}^n \left(\hat{w}_{ml}^T x_i - y_i \right)^2 = f(\hat{w}_{ml})$$

Consider the set of all w s.t.

$$f(w) = f(\hat{w}_{ml}) + c \rightarrow c > 0$$

$$S_c = \{w : f(w) = f(\hat{w}_{ml}) + c\}$$

Every $w \in S_c$ satisfies

$$\|X^T w - y\|^2 = \|X^T \hat{w}_{ml} - y\|^2 + c$$

On Simplification

$$(w - \hat{w}_{ml})^T (XX^T) (w - \hat{w}_{ml}) = c'$$

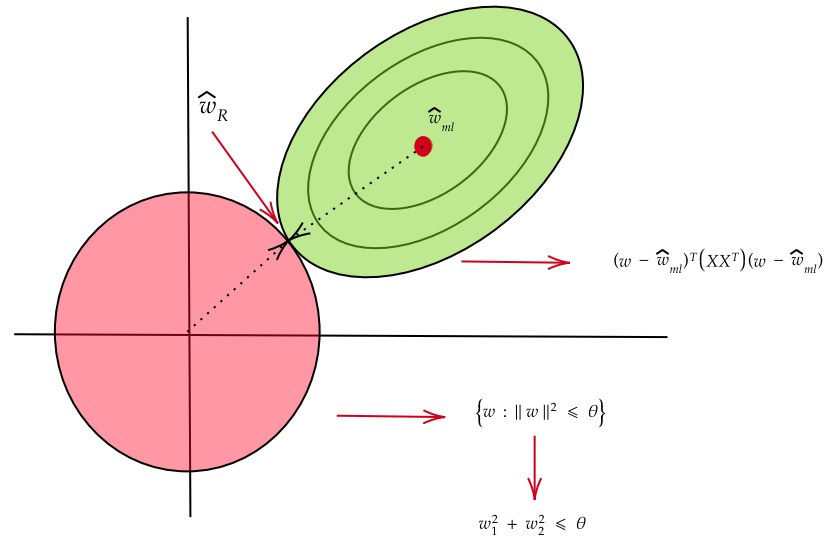
The Value of c' depends on c , XX^T , $\hat{w}_{ml}$ and not on w

If $(XX^T) = I$

$$(w - \hat{w}_{ml})^T (XX^T) (w - \hat{w}_{ml}) = c'$$

$$(w - \hat{w}_{ml})^T I (w - \hat{w}_{ml}) = c'$$

$$\|w - \hat{w}_{ml}\|^2 = c'$$



Conclusion : Ridge Regression pushes weight towards 0, but doesn't necessarily make it 0

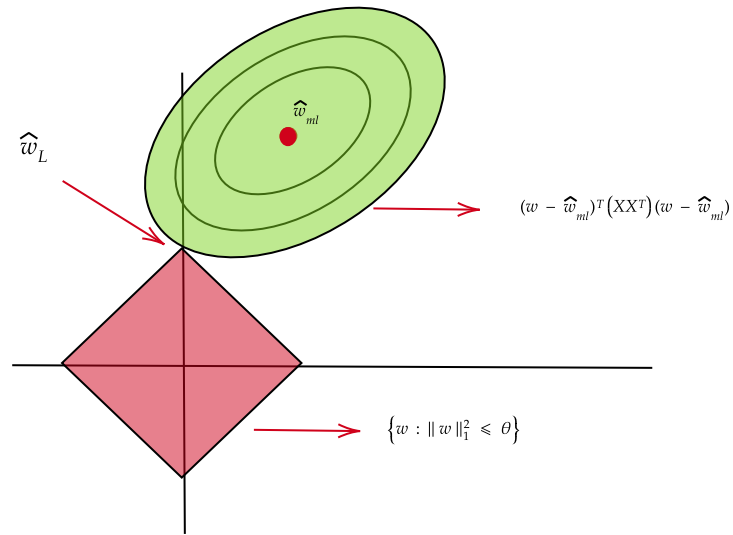
Relation Between Linear Regression and LASSO Regression

An alternate way to regularise would be using $\| \cdot \|_1$ norm instead of $\| \cdot \|_2$ norm

$$\hat{w}_L = \arg \min_w \sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \|w\|_1^2 \quad (2)$$

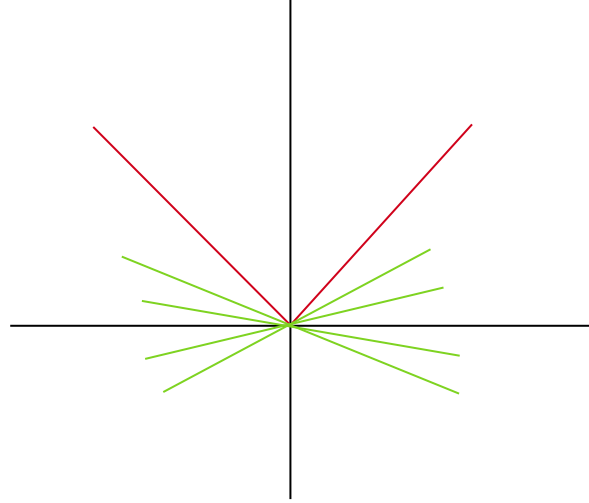
$$\begin{aligned} &\equiv \\ &\min_{w \in \mathbb{R}^d} \sum_{i=1}^n (w^T x_i - y_i)^2 \\ &\quad s. t. \|w\|_1 \leq \theta \end{aligned} \quad (3)$$

Lasso (Least Absolute Shrinkage and Selection Operator) regression is a linear regression technique that employs a regularization approach to shrink the coefficients of less important features to zero. This method effectively performs feature selection and mitigates overfitting.



- LASSO doesn't have closed form solution
- Sub Gradient Methods are usually used to solve LASSO

Sub Gradient



Reference

Arun Raj Kumar, Lecture Slides for Machine Learning Techniques

Karthik Thiagarajan, Machine Learning Techniques Github Notes.

Indian Institute of Technology Madras , CS2007 Course Website

Stanford University, CS229 Machine Learning Notes

Thank You !